

## Maximum Rectangle Area

**Company:** Google

**Round:** Offline

**Year:** 2025

**Difficulty:** Hard

**Topic:** 2D Geometry, Arrays, Optimization

**Source:** <https://www.designa.in/19233/amazon-sde-2-recent-interview-experiences-2026-set-66>

### Problem Description

Given a set of points on a 2D plane, where every point is represented as:

[x, y]

find the **maximum area of a rectangle** that can be formed using the given points.

### Input Format

- The first line contains an integer N, representing the number of points.
- The next N lines contain two integers x and y, representing the coordinates of each point..

### Output Format

Print a single integer representing the **maximum area of a rectangle** that can be formed using the given points.

### Sample Input

```
4
1 1
1 4
5 1
5 4
```

### Sample Output

```
12
```

### Explanation

The four points form a rectangle:

(1,4) ----- (5,4)

|                      |  
|                      |

(1,1) ----- (5,1)

Width:

$$5 - 1 = 4$$

Height:

$$4 - 1 = 3$$

Area:

$$4 \times 3 = 12$$

Therefore, the maximum area is:

12

### Approach to Solve This Problem:

- Store the given points in a suitable data structure for quick checking.
- Consider possible pairs of points as opposite corners.
- Check whether the other two required corners also exist.
- Calculate the rectangle's area.
- Keep track of the maximum area found.
- Optimize the approach to avoid checking unnecessary combinations.

### Java Solution:

```
1 import java.util.*;
2 public class Main {
3     public static int maxRectangleArea(int[][] points) {
4         HashSet<String> set = new HashSet<>();
5         for (int[] p : points) {
6             set.add(p[0] + "," + p[1]);
7         }
8         int maxArea = 0;
9         for (int i = 0; i < points.length; i++) {
10            for (int j = i + 1; j < points.length; j++) {
11                int x1 = points[i][0];
12                int y1 = points[i][1];
13                int x2 = points[j][0];
14                int y2 = points[j][1];
15                if (x1 == x2 || y1 == y2) {
16                    continue;
17                }
18                String p3 = x1 + "," + y2;
19                String p4 = x2 + "," + y1;
20                if (set.contains(p3) && set.contains(p4)) {
21                    int width = Math.abs(x2 - x1);
22                    int height = Math.abs(y2 - y1);
23                    int area = width * height;
24                    maxArea = Math.max(maxArea, area);
25                }
26            }
27        }
28    }
29 }
```

```
27     }
28
29     return maxArea;
30 }
31 public static void main(String[] args) {
32     int[][] points = {
33         {1, 1},
34         {1, 4},
35         {5, 1},
36         {5, 4}
37     };
38     System.out.println(maxRectangleArea(points));
39 }
40 }
```

### Solution:

The main idea is to use a **HashSet** to quickly check whether a point exists.

1. Store all the given points in a HashSet.
2. Select any two points as possible **opposite corners** of a rectangle.
3. Ignore the pair if they have the same x or same y coordinate.
4. Calculate the positions of the other two corners.
5. Check whether both missing corners exist in the HashSet.
6. If they exist, calculate the rectangle's width and height.
7. Calculate the area and keep the **maximum area** found.

### Complexity Analysis:

**Time Complexity:**  $O(N^2)$

We check every possible pair of points to consider them as opposite corners.

**Space Complexity:**  $O(N)$

We store all N points in a HashSet for fast lookup.